Encapsulation and Data Hiding

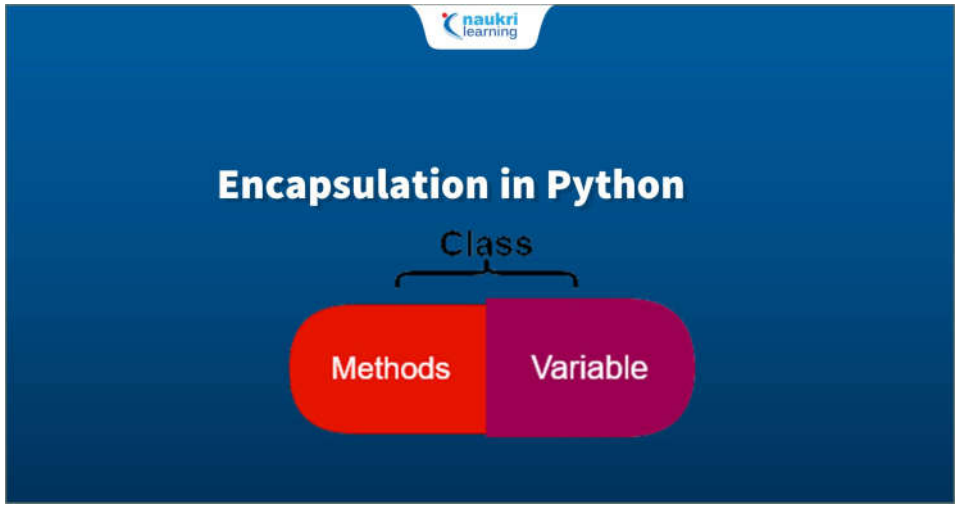
Flow of Discussion

- What is Encapsulation?
- What is Data Hiding
- How Encapsulation used in Industry
- Access Modifiers in Python
- Public Member
- Private Member
  - Public method to access private members
  - Name Mangling to access private members
- Protected Member
- Getters and Setters in Python
- Advantages of Encapsulation

What is Encapsulation?

- Encapsulation, refers to the practice of bundling data (variables) and the methods that operate on that data within a single unit (class), while
- Encapsulation involves wrapping up the data, like code elements, into a single unit.

for example, when you create a class, it means you are implementing encapsulation. A class is an example of encapsulation as it binds all the data members (instance variables) and methods into a single unit.

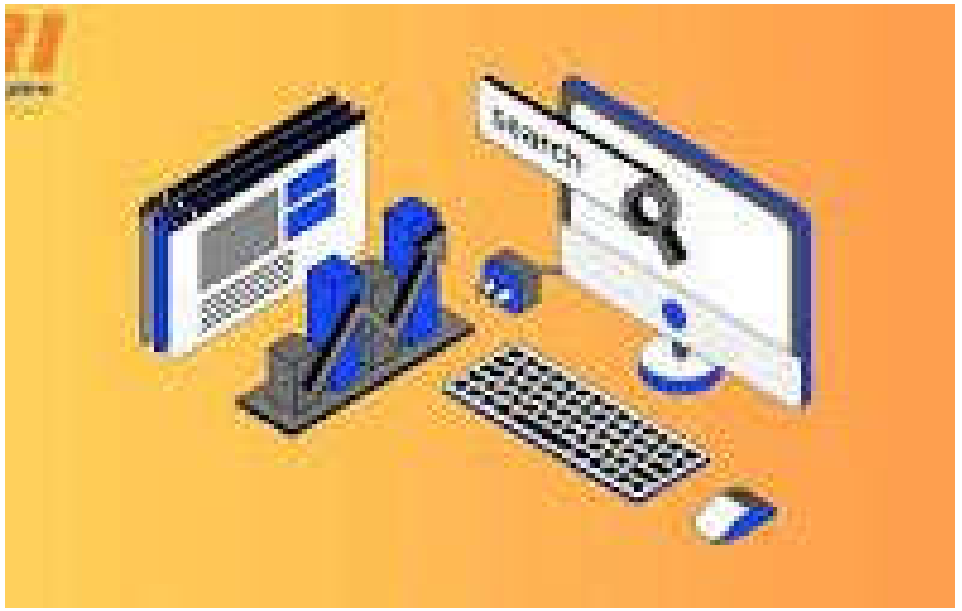


Example:

```
class Employee:
    # constructor
    def __init__(self, name, salary, project):
        # data members
        self.name = name
        self.salary = salary
        self.project = project

    # method
    # to display employee's details
    def show(self):
        # accessing public data member
        print("Name: ", self.name, 'Salary:', self.salary)

    # method
    def work(self):
        print(self.name, 'is working on', self.project)

# creating object of a class
emp = Employee('Jessa', 8000, 'NLP')

# calling public method of the class
emp.show()
emp.work()
```

What is Data Hiding

- Data hiding, is the specific mechanism of restricting direct access to an object's internal data, usually by making data members private and providing controlled access through public methods like getters and setters, thereby protecting data integrity and preventing unauthorized modifications; essentially, data hiding is a key aspect of encapsulation.



Example

```
1 usage
class DataHideExample():
    __secretNum = 99
    notSecretNum = 33333333

    1 usage
    def increase(self):
        self.__secretNum +=1
        print(self.__secretNum)

example = DataHideExample()

example.increase()

print(example.notSecretNum)
```

Key Points to Remember:

Key Differences Between Data Hiding and Encapsulation

1. Encapsulation deals with hiding the **complexity** of a program. On the other hand, data hiding deals with the **security** of data in a program.

2. Encapsulation focuses on **wrapping** (encapsulating) the complex data in order to present a simpler view for the user. On the other hand, data hiding focuses on **restricting** the use of data, intending to assure the data security.

3. In encapsulation data can be **public or private** but, in data hiding, data must be **private** only.

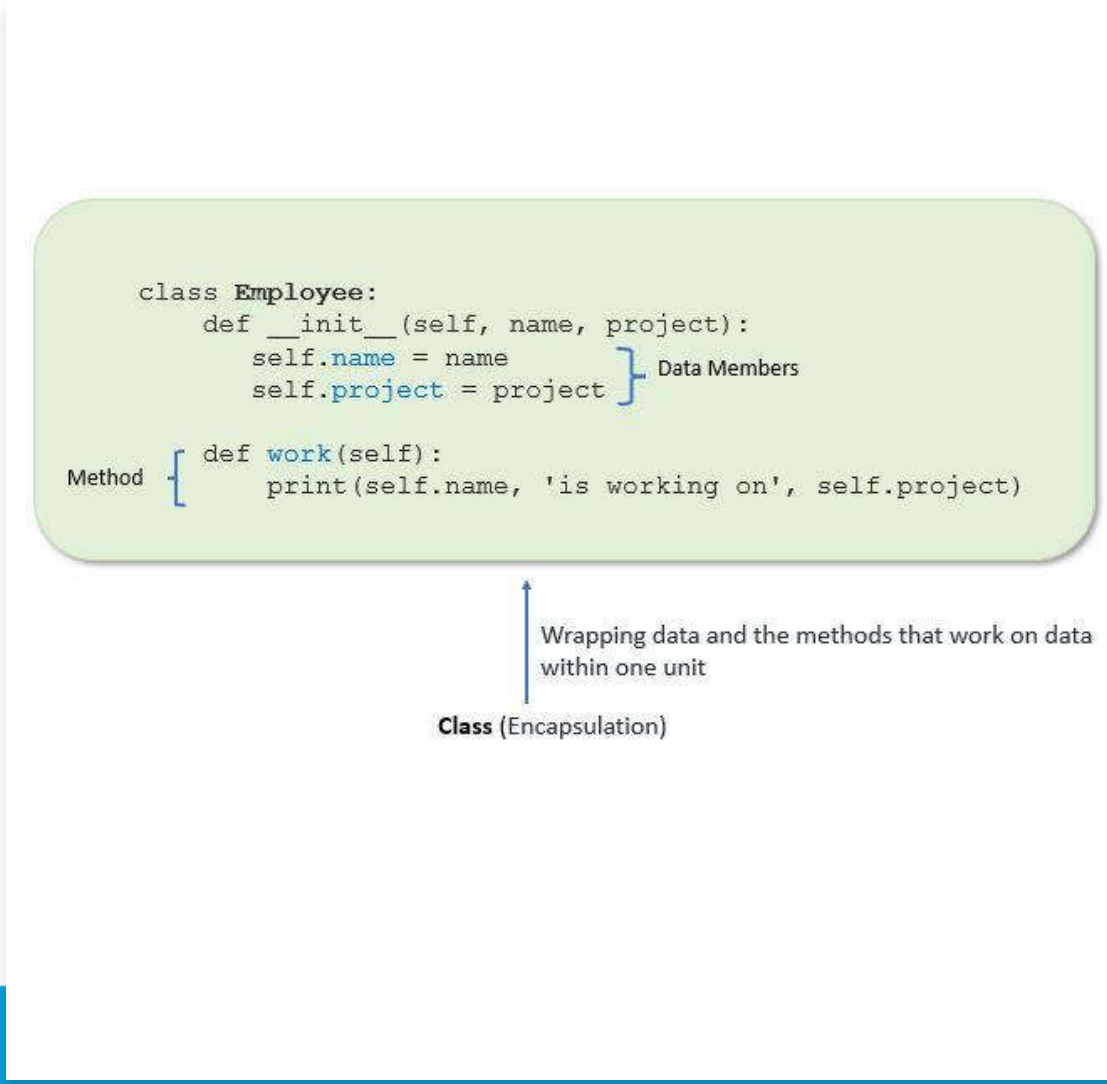
4. Data hiding is a **process** as well as a technique whereas, encapsulation is **subprocess** in data hiding.

Comparison Chart

BASIS FOR COMPARISON	DATA HIDING	ENCAPSULATION
Basic	Data hiding concern about data security along with hiding complexity.	Encapsulation concerns about wrapping data to hide the complexity of a system.
Focus	Data Hiding focuses on restricting or permitting the use of data inside the capsule.	Encapsulation focuses on enveloping or wrapping the complex data.
Access Specifier	The data under data hiding is always private and inaccessible.	The data under encapsulation may be private or public.
Process	Data hiding is a process as well as technique.	Encapsulation is a sub-process in data hiding.

Encapsulation

- Using encapsulation, we can hide an object's internal representation from the outside.
- Encapsulation allows us to restrict accessing variables and methods directly and prevent accidental data modification by creating private data members and methods within a class.



How is Encapsulation used in the industry?

- Encapsulation is used with abstraction to define the level of access you want to give a user. Consider the following:
- **Login Verification**
  - When you log in to check your mail on Gmail, yahoo mail, etc., you enter your password for logging. The password is then encrypted and verified, and access is granted on successful verification.
  - But being a user, you don’t know how your password is being verified in the backend, keeping your account safe. This is one of the many actual cases of how Encapsulation is used in the industry.

How is Encapsulation used in the industry?

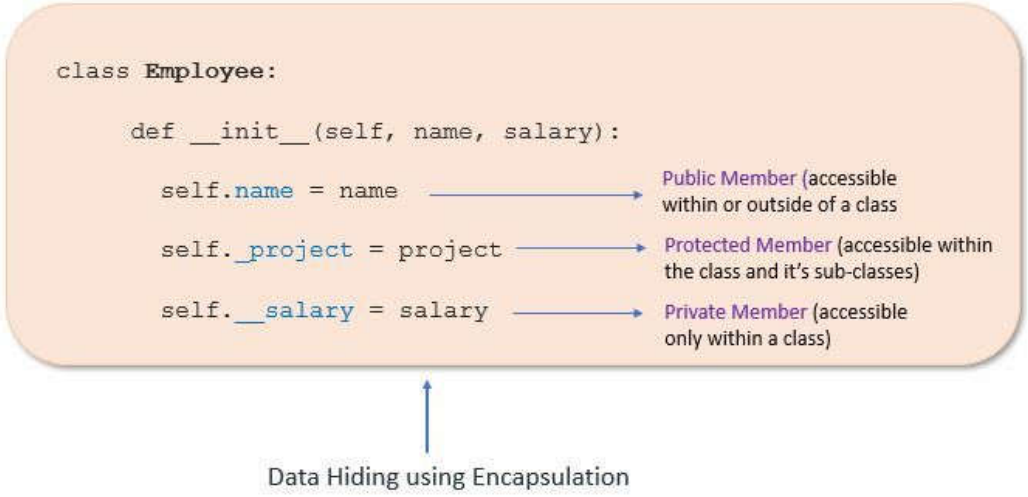
- **Bank Statements Accounts**
  - bank balance to be a piece of public information. This will be the case if the balance variable in the banking app is declared a public variable.
  - Developers declare the balance variable as private to keep the details safe so that no one can see your account balance.
  - The person who wants to check his account balance will be authenticated. Only the authenticated users can access the private members defined inside that class.
  - This private method would be an account verification method that will match your saved account number or userID and password in the database with the entered details (userID and password) for authentication.

Access Modifiers in Python

- Encapsulation can be achieved by declaring the data members and methods of a class either as private or protected.
- But In Python, we don’t have direct access modifiers like public, private, and protected.
- We can achieve this by using single **underscore** and **double underscores**.

Access Modifiers in Python

- Access modifiers limit access to the variables and methods of a class. Python provides three types of access modifiers private, public, and protected.
  - **Public Member**: Accessible anywhere from otside oclass.
  - **Private Member**: Accessible within the class
  - **Protected Member**: Accessible within the class and its sub-classes



Public Member

Public data members are accessible within and outside of a class.

All member variables of the class are by default public.

Example

```
class Employee:
    # constructor
    def __init__(self, name, salary):
        # public data members
        self.name = name
        self.salary = salary

    # public instance methods
    def show(self):
        # accessing public data member
        print("Name: ", self.name, 'Salary:', self.salary)

# creating object of a class
emp = Employee('Jessa', 10000)

# accessing public data members
print("Name: ", emp.name, 'Salary:', emp.salary)

# calling public method of the class
emp.show()
```

Private Member

- We can protect variables in the class by marking them private. To define a private variable, add two underscores as a prefix at the start of a variable name.
- In this example, the salary is a private variable. As you know, we can’t access the private variable from the outside of that class.
- We can access private members from outside of a class using the following two approaches
  - Create public method to access private members
  - Use name mangling

Example

```
class Employee:
    # constructor
    def __init__(self, name, salary):
        # public data member
        self.name = name
        # private member
        self.__salary = salary

# creating object of a class
emp = Employee('Jessa', 10000)

# accessing private data members
print('Salary:', emp.__salary)
```

Public Method to Access Private Members

- Access Private member outside of a class using an instance method

Example

```
class Employee:
    # constructor
    def __init__(self, name, salary):
        # public data member
        self.name = name
        # private member
        self.__salary = salary

    # public instance methods
    def show(self):
        # private members are accessible from a class
        print("Name: ", self.name, 'Salary:', self.__salary)

# creating object of a class
emp = Employee('Jessa', 10000)

# calling public method of the class
emp.show()
```

Name Mangling to Access Private Members

- We can directly access private and protected variables from outside of a class through name mangling.
- The name mangling is created on an identifier by adding two leading underscores and one trailing underscore, like this _classname__dataMember, where classname is the current class, and data member is the private variable name.

Example

```python
1 usage
class Example:
    def __init__(self):
        self.__private_var = "I am a private variable"

    1 usage
    def __private_method(self):
        return "This is a private method"

    def access_private_var(self):
        return self.__private_var  # Accessing inside the class

    def access_private_method(self):
        return self.__private_method()  # Accessing inside the class

# Create an object of Example class
obj = Example()

# Trying to access private attributes directly will raise an AttributeError
try:
    print(obj.__private_var)
except AttributeError:
    print("Cannot access __private_var directly!")

try:
    print(obj.__private_method())
except AttributeError:
    print("Cannot access __private_method directly!")

# Accessing private attributes using name mangling
print(obj._Example__private_var)  # Name-mangled variable
print(obj._Example__private_method())  # Name-mangled method
```

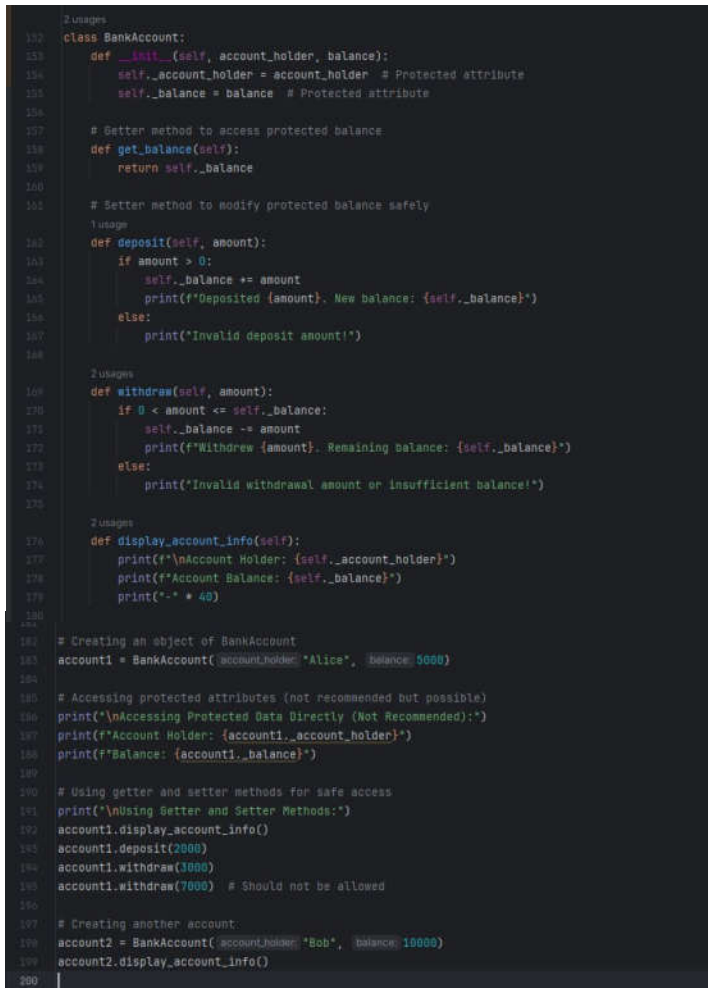
Example:

- We can protect variables in the class by marking them private. To make a classname a mangling with one _underscore, variable a private just add two underscores as a prefix at the start of its name. Example _Employee__salary

```
class Employee:
    # constructor
    def __init__(self, name, salary):
        # public data member
        self.name = name
        # private member
        self.__salary = salary

# creating object of a class
emp = Employee('Jessa', 10000)

print('Name:', emp.name)
# direct access to private member using name mangling
print('Salary:', emp._Employee__salary)
```

Protected Member

- Protected members are accessible within the class and also available to its sub-classes. To define a protected member, prefix the member's name with a single underscore _.
- Protected data members are used when you implement inheritance and want to allow data members access to only child classes.



```
2 usages
class BankAccount:
    def __init__(self, account_holder, balance):
        self._account_holder = account_holder  # Protected attribute
        self._balance = balance  # Protected attribute

    # Getter method to access protected balance
    def get_balance(self):
        return self._balance

    # Setter method to modify protected balance safely
    1 usage
    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
            print(f"Deposited {amount}. New balance: {self._balance}")
        else:
            print("Invalid deposit amount!")

    2 usages
    def withdraw(self, amount):
        if 0 < amount <= self._balance:
            self._balance -= amount
            print(f"Withdrew {amount}. Remaining balance: {self._balance}")
        else:
            print("Invalid withdrawal amount or insufficient balance!")

    2 usages
    def display_account_info(self):
        print(f"\nAccount Holder: {self._account_holder}")
        print(f"Account Balance: {self._balance}")
        print("-" * 40)


# Creating an object of BankAccount
account1 = BankAccount( account_holder: "Alice",  balance: 5000)

# Accessing protected attributes (not recommended but possible)
print("\nAccessing Protected Data Directly (Not Recommended):")
print(f"Account Holder: {account1._account_holder}")
print(f"Balance: {account1._balance}")

# Using getter and setter methods for safe access
print("\nUsing Getter and Setter Methods:")
account1.display_account_info()
account1.deposit(2000)
account1.withdraw(3000)
account1.withdraw(7000)  # Should not be allowed

# Creating another account
account2 = BankAccount( account_holder: "Bob",  balance: 10000)
account2.display_account_info()
```

Example

- Protected members are accessible within the class and also available to. To define a protected member, prefix the member's name with a single underscore.

Getters and Setters

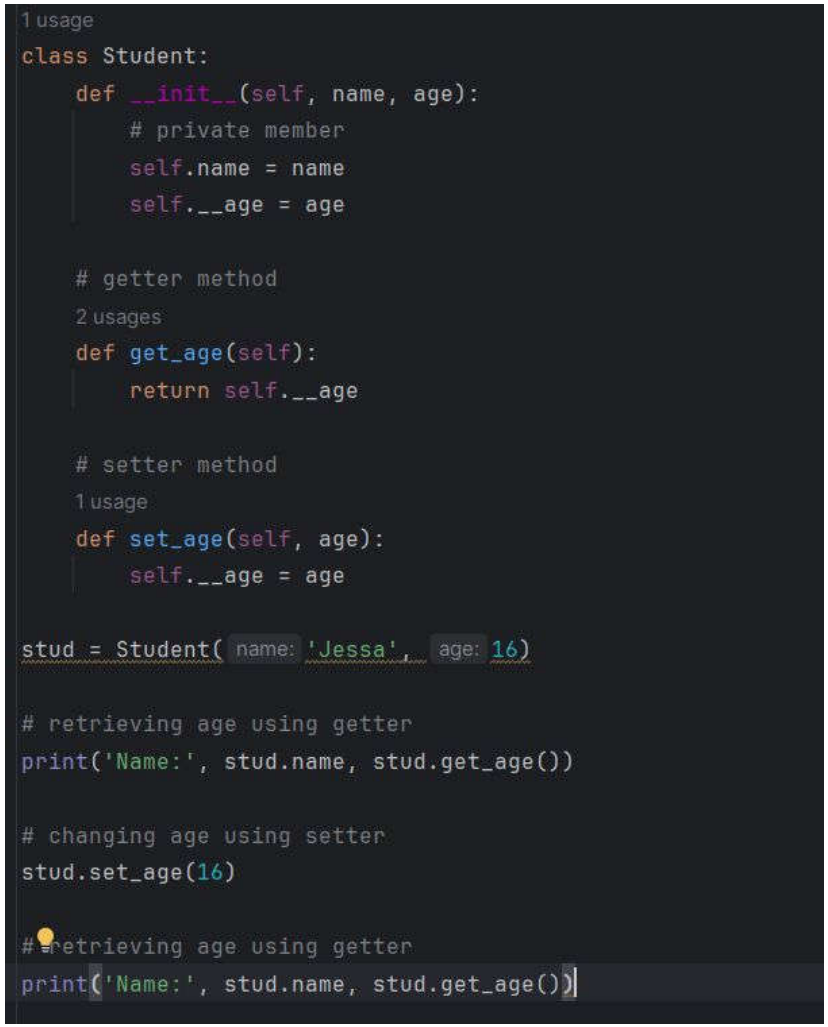
- To implement proper encapsulation in Python, we need to use setters and getters. The primary purpose of using getters and setters in object-oriented programs is to ensure data encapsulation.
  - **Getters**: These are the methods used in Object-Oriented Programming (OOPS) which helps to access the private attributes from a **class**. Getters are methods used to access or 'get' the value of an object's attributes. They provide a way to read data without directly accessing the attributes.
  - **Setters**: These are the methods used in OOPS feature which helps to set the value to private attributes in a **class**. Setters are methods used to change or 'set' the value of an object's attributes. They allow for data modification while implementing checks or validations.

Example

```python
1 usage
class Employee:
    def __init__(self, name, salary):
        self._name = name          # Protected attribute
        self._salary = salary      # Protected attribute

    1 usage
    @property
    def name(self):
        """Getter method for name"""
        return self._name

    1 usage
    @property
    def salary(self):
        """Getter method for salary"""
        return self._salary

# Create an Employee object
emp = Employee( name: "John Doe",  salary: 50000)

# Access attributes using getters
print("Employee Name:", emp.name)
print("Employee Salary:", emp.salary)
```

Example

- Implement information hiding and apply additional validation before changing the values of your object attributes (data member).



```
1 usage
class Student:
    def __init__(self, name, age):
        # private member
        self.name = name
        self.__age = age

    # getter method
    2 usages
    def get_age(self):
        return self.__age

    # setter method
    1 usage
    def set_age(self, age):
        self.__age = age

stud = Student( name: 'Jessa',  age: 16)

# retrieving age using getter
print('Name:', stud.name, stud.get_age())

# changing age using setter
stud.set_age(16)

# retrieving age using getter
print('Name:', stud.name, stud.get_age())
```

Example

- Information Hiding and with conditional logic for setting an object attributes

```python
class Student:
    def __init__(self, name, roll_no, age):
        # private member
        self.name = name
        # private members to restrict access
        # avoid direct data modification
        self.__roll_no = roll_no
        self.__age = age

    2 usages
    def show(self):
        print('Student Details:', self.name, self.__roll_no)

    # getter methods
    def get_roll_no(self):
        return self.__roll_no

    # setter method to modify data member
    # condition to allow data modification with rules
    2 usages
    def set_roll_no(self, number):
        if number > 50:
            print('Invalid roll no. Please set correct roll number')
        else:
            self.__roll_no = number

jessa = Student( name: 'Jessa', roll_no: 10, age: 15)

# before Modify
jessa.show()
# changing roll number using setter
jessa.set_roll_no(120)


jessa.set_roll_no(25)
jessa.show()
```

Advantages of Encapsulation

1. **Data Protection:** Encapsulation protects data from unauthorized access and modification. It enforces access controls on class members (attributes and methods) to define who can access the data.

   Example: Private attributes and methods are only accessible within the class, ensuring data integrity.

2. **Modular Code Design:** Encapsulation promotes a modular code design by bundling related data and functionality within a class. This makes the code easier to manage, maintain, and extend.

   Example: Grouping attributes and methods related to a specific entity, like an employee or a car, within a class.

3. **Abstraction:** Encapsulation abstracts complex internal details, providing a simplified and understandable interface for interacting with a class. External code doesn't need to know how things work internally.

   Example: A Car class offers methods like start and stop to interact with the car's engine without exposing intricate engine details.

4. **Controlled Access:** Encapsulation allows controlled access to data and methods through well-defined interfaces. It reduces the risk of unintended changes and errors by enforcing data validation and access control rules.

   Example: Methods like set_temperature in a TemperatureConverter class ensure that temperature values stay within valid ranges.

Advantages of Encapsulation

5. **Code Reusability**: Encapsulation promotes code reusability. Well-encapsulated classes can be easily integrated into different parts of an application, reducing duplication and improving code consistency.

   Example: Using a Person class to manage user data consistently in various modules of an application.

6. **Improved Security:** Encapsulation enhances security by controlling access to sensitive data. Private members are inaccessible from outside the class, reducing the risk of data breaches or misuse.

   Example: Keeping passwords or encryption keys as private attributes within a class.

7. **Encapsulation Hierarchies:** Inheritance and encapsulation can be combined to create encapsulation hierarchies. Subclasses can inherit and extend the encapsulation of their parent classes.

   Example: A base Shape class encapsulates common attributes and methods, while subclasses like Circle and Rectangle encapsulate shape-specific details.

8. **Code Readability and Maintenance:** Encapsulation enhances code readability and maintainability by providing a clear structure and well-defined interfaces. It simplifies troubleshooting and debugging.

   Example: Well-encapsulated classes are easier to understand and maintain over time.

Disadvantages of Encapsulation in Python

1. Complexity: Overusing encapsulation can lead to code complexity and verbosity, making the code harder to understand.

Example: Extremely fine-grained encapsulation with numerous getters and setters can clutter the codebase.

2. Performance Overhead: Accessing data through methods can introduce a slight performance overhead compared to direct attribute access.

Example: In high-performance applications, frequent method calls for simple attribute access can impact speed.

3. Name Mangling: The use of name mangling to access private members can lead to less readable and maintainable code.

Example: Accessing private members using name mangling involves complex naming conventions and may confuse developers.

Disadvantages of Encapsulation in Python

4. Limited Flexibility: Overly strict encapsulation can limit flexibility and make it challenging to extend or modify a class.

Example: If a class encapsulates data and operations too tightly, making changes in the class requires extensive modifications.

5. Complexity in Testing: Testing encapsulated code can be more complex, as you need to write additional test methods to access encapsulated data.

Example: In unit testing, you have to create extra methods to set encapsulated data for testing purposes.

Benefits of Encapsulation and Data Hiding:

- **Improved code maintainability:** By hiding implementation details, changes within a class are less likely to impact other parts of the program.
- **Enhanced data security:** Restricting direct access to data protects it from unauthorized modifications.
- **Modularity:** Encapsulation promotes the creation of well-defined, independent objects.


END